
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 4

Controle de Programa em C

Exercícios de Autorrevisão (4.1–4.4)

Resolvidos passo a passo, abordagem incremental

Capítulos 1 + 2 + 3 + 4

Construindo sobre os Capítulos Anteriores

No Capítulo 4, ampliamos significativamente o controle de programa em C. Construindo sobre os fundamentos dos Capítulos 1, 2 e 3, incorporamos:

- **Estrutura `for`** – ideal para repetição controlada por contador
- **Estrutura `do...while`** – testa a condição *depois* do corpo
- **Estrutura `switch`** – seleção múltipla baseada em valores constantes
- **`break` e `continue`** – modificadores do fluxo de loops
- **Operadores lógicos `&&` (E), `||` (OU), `!` (NÃO)**
- **Diferença entre `=` e `==`** – atribuição vs. comparação
- **Função `pow`** da biblioteca matemática `<math.h>`
- **Formatação avançada de `printf`** – largura de campo e alinhamento

Boa prática de programação

Os recursos do Capítulo 4 dão ao seu programa muito mais expressividade. `for` é a forma idiomática de loops contados; `switch` substitui cadeias de `if...else` quando comparamos uma variável com valores constantes; e os operadores lógicos permitem combinar condições naturalmente.

Contents

1	Exercício 4.1 – Preenchimento de Lacunas	2
2	Exercício 4.2 – Verdadeiro ou Falso	5
3	Exercício 4.3 – Instruções com novos recursos	7
4	Exercício 4.4 – Identificar e Corrigir Erros	10

1. Exercício 4.1 – Preenchimento de Lacunas

Enunciado 4.1

Preencha os espaços nas seis sentenças sobre repetição controlada por contador, repetição controlada por sentinela, comandos de controle e estrutura `switch`.

Item (a) – Repetição controlada por contador

Resposta detalhada

Resposta: definida

A repetição controlada por contador é também conhecida como *repetição definida*, porque o número de iterações é **conhecido com antecedência**, antes que o laço comece a ser executado. Por exemplo, “execute esta tarefa exatamente 10 vezes”. A estrutura `for` foi projetada especificamente para esse padrão.

Item (b) – Repetição controlada por sentinela

Resposta detalhada

Resposta: indefinida

A repetição controlada por sentinela é também conhecida como *repetição indefinida*, porque **não sabemos com antecedência** quantas vezes o laço executará. O usuário controla isso digitando o valor de sentinela quando desejar terminar.

Item (c) – O elemento contador da repetição definida

Resposta detalhada

Resposta: variável de controle (ou contador)

A *variável de controle* é a variável que conta o número de iterações executadas. Em uma estrutura `for`, ela aparece três vezes no cabeçalho:

```
1  for ( i = 1; i <= 10; ++i ) {  
2      /* corpo do for */  
3  }
```

- `i = 1` – valor *inicial* da variável de controle
- `i <= 10` – condição de continuação testada *antes* de cada iteração
- `++i` – incremento da variável de controle, após cada iteração

Item (d) – Comando para a próxima iteração

Resposta detalhada

Resposta: continue

O comando `continue` pula o restante do corpo do laço e prossegue imediatamente com a próxima iteração. Em `while` e `do...while`, isso significa retornar ao teste da condição; em `for`, executa primeiro a expressão de incremento.

```
1  /* Imprime 1 a 10, mas pula 5 */
2  for ( i = 1; i <= 10; ++i ) {
3      if ( i == 5 ) {
4          continue; /* pula esta iteracao */
5      }
6      printf( "%d ", i );
7  }
8  /* Saída: 1 2 3 4 6 7 8 9 10 */
```

Item (e) – Comando para sair imediatamente da estrutura

Resposta detalhada

Resposta: break

O comando `break` causa a saída **imediate** de uma estrutura de repetição (`for`, `while`, `do...while`) ou de uma estrutura `switch`. A execução continua na primeira instrução *após* a estrutura.

```
1  /* Encontra o primeiro multiplo de 7 acima de 50 */
2  for ( i = 50; i <= 100; ++i ) {
3      if ( i % 7 == 0 ) {
4          printf( "Encontrado: %d\n", i );
5          break; /* sai do for imediatamente */
6      }
7  }
```

Item (f) – Estrutura para testar valores inteiros constantes

Resposta detalhada

Resposta: estrutura de seleção múltipla switch

A estrutura `switch` (Seção 4.7) testa uma variável (ou expressão) contra valores inteiros constantes – chamados *rótulos case*. Para cada rótulo correspondente, executa as instruções associadas até encontrar um `break`. O caso `default` (opcional) captura todos os valores não casados.

```
1  switch ( opcao ) {
2      case 1:
3          printf( "Opcao 1 escolhida\n" );
```

```
4         break;
5     case 2:
6         printf( "Opcao 2 escolhida\n" );
7         break;
8     default:
9         printf( "Opcao invalida\n" );
10        break;
11    }
```

2. Exercício 4.2 – Verdadeiro ou Falso

Item (a) – O caso default é obrigatório no switch

FALSO

O caso **default** é **opcional**. Se não for necessário tomar nenhuma ação para valores não casados pelos rótulos **case**, o **default** pode ser omitido. Sem **default**, valores não correspondentes simplesmente não disparam nenhuma ação.

Boa prática de programação

Mesmo quando aparentemente desnecessário, fornecer um **default** é uma excelente prática defensiva. Ele captura valores inesperados (devido a bugs ou entradas inválidas) que poderiam passar silenciosamente despercebidos.

Item (b) – O break é obrigatório no caso default

FALSO

O **break** é **usado para sair da estrutura switch**, evitando o efeito conhecido como *fall-through* (queda em cascata para o próximo **case**). Quando o **default** é o *último* rótulo do **switch**, o **break** é desnecessário, pois não há mais casos para os quais “cair”.

Ainda assim, é uma boa prática colocar **break** no **default**, pois se um **case** adicional for adicionado depois, no futuro, o **break** já estará lá prevenindo bugs.

Item (c) – A expressão (x > y && a < b)

FALSO

O operador **&&** (E lógico) exige que **ambas** as expressões relacionais sejam verdadeiras para que o resultado total seja verdadeiro. A enunciação “ou” do enunciado descreve o operador **||** (OU lógico), não o **&&**.

Tabela-verdade do &&:

x > y	a < b	x > y && a < b
falso	falso	falso
falso	verdadeiro	falso
verdadeiro	falso	falso
verdadeiro	verdadeiro	verdadeiro

Item (d) – A expressão com ||

VERDADEIRO

O operador || (OU lógico) é verdadeiro se **pelo menos um** dos operandos for verdadeiro. Em particular, é verdadeiro também quando *ambos* são verdadeiros. Apenas quando *ambos* são falsos é que o || produz falso.

3. Exercício 4.3 – Instruções com novos recursos

Item (a) – Soma dos ímpares de 1 a 99 com for

Resposta detalhada

```
1 soma = 0;
2 for ( contador = 1; contador <= 99; contador += 2 ) {
3     soma += contador;
4 }
```

Explicação: Inicializamos `contador` em 1 (primeiro ímpar) e incrementamos de 2 em 2 (`contador += 2`), garantindo que percorremos apenas ímpares: 1, 3, 5, ..., 99. **Resultado:** $1 + 3 + 5 + \dots + 99 = 2500$.

Item (b) – Imprimir 333,546372 com 5 precisões diferentes

Resposta detalhada

```
1 printf( "%-15.1f\n", 333.546372 ); /* imprime: 333.5
   */
2 printf( "%-15.2f\n", 333.546372 ); /* imprime: 333.55
   */
3 printf( "%-15.3f\n", 333.546372 ); /* imprime:
   333.546 */
4 printf( "%-15.4f\n", 333.546372 ); /* imprime:
   333.5464 */
5 printf( "%-15.5f\n", 333.546372 ); /* imprime:
   333.54637 */
```

Explicação dos especificadores:

- % – início do especificador
- - – alinha à **esquerda** no campo
- 15 – largura do campo (15 posições)
- .N – precisão (N casas decimais)
- f – tipo float ou double

Os 5 valores impressos: 333.5, 333.55, 333.546, 333.5464, 333.54637.
Note o arredondamento, não truncamento.

Item (c) – Calcular $2,5^3$ usando pow

Resposta detalhada

```
1 #include <math.h> /* necessario para usar pow */
2 /* ... */
3 printf( "%10.2f\n", pow( 2.5, 3 ) ); /* imprime: "
   15.63" */
```

Valor impresso: 15.63 (com 6 espaços antes, totalizando 10 posições).

Observações:

- `pow(base, expoente)` retorna $base^{expoente}$ como `double`
- $2,5^3 = 2,5 \times 2,5 \times 2,5 = 15,625 \rightarrow$ arredondado a 2 casas: 15.63
- Sem o `-`, o valor é alinhado à **direita** no campo de 10 caracteres

Boa prática de programação

Para usar funções da biblioteca matemática (`pow`, `sqrt`, `sin`, etc.), inclua `<math.h>`. Em sistemas Linux, ao compilar, pode ser necessário ligar a biblioteca matemática com `-lm`: `gcc programa.c -o programa -lm`.

Item (d) – Imprimir 1 a 20 com `while`, 5 por linha

Resposta detalhada

```
1  x = 1;
2  while ( x <= 20 ) {
3      printf( "%d", x );
4      if ( x % 5 == 0 ) {
5          printf( "\n" );    /* nova linha a cada 5
                               numeros */
6      }
7      else {
8          printf( "\t" );    /* tabulacao entre numeros
                               */
9      }
10     ++x;
11 }
```

Saída esperada:

Saída do programa

```
1\t2\t3\t4\t5
6\t7\t8\t9\t10
11\t12\t13\t14\t15
16\t17\t18\t19\t20
```

Lógica do truque: `x % 5 == 0` é verdadeiro apenas para múltiplos de 5 (5, 10, 15, 20) – exatamente onde queremos quebrar a linha.

Item (e) – O mesmo, com `for`

Resposta detalhada

```
1  for ( x = 1; x <= 20; ++x ) {
2      printf( "%d", x );
3      if ( x % 5 == 0 ) {
4          printf( "\n" );
```

```
5     }  
6     else {  
7         printf( "\\t" );  
8     }  
9 }
```

Comparação while vs. for: A versão `for` é mais *idiomática* para repetição controlada por contador. As três partes do controle (inicialização, condição, incremento) ficam concentradas no cabeçalho do `for`, tornando o código mais conciso e legível.

4. Exercício 4.4 – Identificar e Corrigir Erros

Item (a) – Loop infinito por ; extra

Resposta detalhada

Código com erro:

```
1 x = 1;  
2 while ( x <= 10 );  
3     x++;  
4 }
```

Erros identificados:

- a) O ponto e vírgula após `while ( x <= 10 )` cria um corpo de laço *vazio*; como `x` nunca é incrementado, gera-se um **loop infinito**.
- b) Falta a chave de abertura `{` (há `}` no final, mas sem `{`).

Correção:

```
1 x = 1;  
2 while ( x <= 10 ) { /* remove o ; e abre chave aqui  
3     x++;  
4 }
```

Item (b) – Ponto flutuante como variável de controle

Resposta detalhada

Código com erro:

```
1 for ( y = .1; y != 1.0; y += .1 )
2     printf( "%f\n", y );
```

Erro: Usar um número de **ponto flutuante** como variável de controle de um **for** é *perigosíssimo*. Devido à imprecisão da representação binária de números fracionários (você não consegue representar 0,1 exatamente em binário, da mesma forma que não consegue representar 1/3 exatamente em decimal), a soma cumulativa de incrementos 0.1 **não chega** exatamente a 1.0, e a condição `y != 1.0` nunca se torna verdadeira. **Loop infinito!**

Correção: use um inteiro como variável de controle e faça a divisão para obter o valor fracionário desejado.

```
1 for ( y = 1; y != 10; y++ )
2     printf( "%f\n", (float) y / 10 );
```

Dica de prevenção de erro

Regra de ouro: *nunca* compare valores de ponto flutuante por igualdade exata (`==` ou `!=`). Se for absolutamente necessário, teste se a diferença é menor que uma pequena tolerância: `fabs(x - y) < 1e-9`.

Item (c) – Falta break no switch

Resposta detalhada

Código com erro:

```
1 switch ( n ) {
2     case 1:
3         printf( "O numero e 1\n" );
4     case 2:
5         printf( "O numero e 2\n" );
6         break;
7     default:
8         printf( "O numero nao e nem 1 nem 2\n" );
9         break;
10 }
```

Erro: Falta o **break** no final do **case 1**. Quando `n == 1`, o programa imprime “O numero e 1” e **continua executando** as instruções do **case 2** (*fall-through*), imprimindo também “O numero e 2” – comportamento incorreto.

Correção:

```
1 switch ( n ) {
2     case 1:
3         printf( "O numero e 1\n" );
4         break; /* adiciona break para evitar fall-
               through */
```

```
5     case 2:
6         printf( "0 numero e 2\n" );
7         break;
8     default:
9         printf( "0 numero nao e nem 1 nem 2\n" );
10        break;
11 }
```

Nota: Em raras situações, o *fall-through* é **intencional**. Por exemplo, na contagem de notas (Figura 4.7 do livro), `case 'A'` cai em `case 'a'` para tratar maiúscula e minúscula igualmente.

Item (d) – Operador relacional incorreto

Resposta detalhada

Código com erro:

```
1  n = 1;
2  while ( n < 10 )
3      printf( "%d ", n++ );
```

Erro: A condição `n < 10` é *estritamente menor que*, então o laço termina quando `n` chega a 10 – imprimindo apenas de 1 a 9, não 1 a 10.

Correção: use `<=` em vez de `<`.

```
1  n = 1;
2  while ( n <= 10 )
3      printf( "%d ", n++ );
```

Erro comum de programação

Erros de **off-by-one** (“erro de uma posição”) – usar `<` em vez de `<=` ou vice-versa – são entre os mais comuns e perigosos. Eles podem fazer um laço executar uma vez a menos (ou a mais) do que deveria, com resultados sutis difíceis de detectar.

Gabarito Rápido – Capítulo 4: Autorrevisão

Respostas Resumidas

4.1:

- a) definida b) indefinida c) variável de controle (contador)
- d) continue e) break f) switch

4.2 – Verdadeiro/Falso:

- a) **Falso** – `default` é opcional
- b) **Falso** – `break` pode ser dispensado se `default` for o último
- c) **Falso** – `&&` exige ambas verdadeiras
- d) **Verdadeiro** – `||` é verdadeiro se um ou ambos forem verdadeiros

4.3: Soluções com `for`, `while`, `pow` e formatação `%-15.Nf`.

4.4:

- a) ; extra após `while` cria loop infinito
- b) Não use `float` como controle de `for`
- c) Falta `break` no `case 1`
- d) Use `<=` em vez de `<`